

R como solución de Business Intelligence

¿Qué es R?

R es un lenguaje de programación y entorno de software libre especializado en **análisis estadístico**, **minería de datos** y **visualización de información**. Fue desarrollado inicialmente por Ross Ihaka y Robert Gentleman en la Universidad de Auckland, Nueva Zelanda, y hoy en día es mantenido por el **R Core Team** y una comunidad activa de más de dos millones de usuarios en todo el mundo.

R se distribuye bajo la licencia **GNU General Public License**, lo que lo convierte en una herramienta **completamente gratuita y de código abierto**, sin restricciones de uso comercial.

Ventajas de usar R como solución BI

1. Gratuito y de código abierto

- Sin costos de licencias, a diferencia de herramientas propietarias como SAS, SPSS o Tableau.
- Acceso al código fuente, lo que permite auditoría, personalización y transparencia total en los algoritmos utilizados.

2. Amplio ecosistema de paquetes (CRAN)

- El **Comprehensive R Archive Network (CRAN)** alberga más de **20,000 paquetes** desarrollados por la comunidad.
- Paquetes clave para BI:
 - **dplyr** / **data.table** — manipulación y transformación de datos.
 - **ggplot2** — visualización de datos de alto nivel (gramática de gráficos).
 - **tidyr** — limpieza y reestructuración de datos.
 - **shiny** — creación de dashboards y aplicaciones web interactivas.
 - **rmarkdown** — generación de reportes dinámicos en HTML, PDF y Word.
 - **plotly** — gráficos interactivos.
 - **DBI** / **odbc** — conexión a bases de datos relacionales.

3. Capacidades estadísticas y analíticas avanzadas

- R fue diseñado desde sus orígenes para el análisis estadístico, lo que le otorga una ventaja frente a herramientas de BI puramente visuales.
- Modelos soportados de forma nativa:
 - Regresiones lineales y logísticas.
 - Series temporales (ARIMA, Prophet, etc.).

- Análisis de clustering (k-means, jerárquico).
- Árboles de decisión y random forests.
- Análisis de componentes principales (PCA).
- Pruebas de hipótesis (t-test, ANOVA, chi-cuadrado).

4. Visualización de datos de alta calidad

- Con **ggplot2** se pueden crear gráficos publicables con un control fino sobre cada elemento visual.
- Integración con librerías interactivas como **plotly**, **highcharter** y **leaflet** (mapas).
- Posibilidad de generar dashboards completos con **shiny** y **flexdashboard**.

5. Reproducibilidad y reportes automatizados

- **R Markdown** y **Quarto** permiten combinar código, texto explicativo y resultados en un solo documento.
- Los análisis pueden ejecutarse de forma programada, generando reportes actualizados automáticamente.
- Ideal para auditoría y cumplimiento normativo, ya que cada paso del análisis queda documentado.

6. Integración con múltiples fuentes de datos

- R puede conectarse a:
 - Bases de datos SQL (MySQL, PostgreSQL, SQL Server, Oracle) mediante **DBI** y **odbc**.
 - Archivos planos (CSV, Excel, JSON, XML).
 - APIs REST y web scraping (**httr**, **rvest**, **jsonlite**).
 - Big Data: **sparklyr** (Apache Spark), **arrow** (formatos columnar).
 - Almacenes en la nube: AWS S3, Google Cloud Storage, Azure Blob.

7. Comunidad activa y soporte continuo

- Foros como **Stack Overflow**, **RStudio Community** y **R-bloggers** ofrecen soporte constante.
- Múltiples conferencias anuales (useR!, RStudio Conference, LatinR).
- Documentación extensa y tutoriales disponibles en múltiples idiomas.

8. Curva de aprendizaje accesible

- Aunque es un lenguaje de programación, su sintaxis está orientada a facilitar operaciones con datos.
- Entornos como **RStudio** (hoy **Posit**) proporcionan una interfaz amigable con editor, consola, visor de gráficos y gestor de archivos integrados.

Comparativa con otras herramientas BI

Característica	R	Power BI / Tableau	Python (con pandas)
Costo	Gratuito	Licencias costosas	Gratuito
Análisis estadístico	Excelente (nativo)	Limitado	Excelente
Visualización	Muy buena	Excelente (interactiva)	Buena
Curva de aprendizaje	Moderada	Baja	Moderada-alta
Reproducibilidad	Alta	Media	Alta
Integración con bases de datos	Muy buena	Buena	Muy buena
Dashboards interactivos	Shiny / Flexdashboard	Nativo	Dash / Streamlit

Flujo de trabajo típico de BI con R

Extraccion (lectura de datos)	->	Limpieza y transformacion (dplyr, tidyr)	->	Analisis estadistico y modelado (modelos predictivos)	->	Visualizacion y reportes (ggplot2, Shiny, R Markdown)
-------------------------------------	----	--	----	---	----	---

Ejemplo mínimo de análisis BI con R

```
# Cargar librerías
library(dplyr)
library(ggplot2)

# Leer datos
ventas <- read.csv("ventas.csv")

# Resumen por región
resumen <- ventas %>%
  group_by(region) %>%
  summarise(
    total_ventas = sum(monto),
```

```

    promedio      = mean(monto),
    transacciones = n()
  )

# Visualización
ggplot(resumen, aes(x = region, y = total_ventas, fill = region)) +
  geom_col() +
  labs(title = "Ventas totales por región", x = "Región", y = "Total ventas") +
  theme_minimal()

```

Trabajo colaborativo con R y GitHub

En un entorno BI real, los análisis rara vez los realiza una sola persona. Los equipos de datos necesitan colaborar sobre el mismo código, compartir reportes, revisar cambios y mantener un historial de todas las transformaciones y modelos desarrollados. Aquí es donde **GitHub** se convierte en un pilar fundamental de la solución BI con R.

1. Control de versiones para análisis de datos

- **Git** permite llevar un historial completo de cada cambio en scripts `.R`, reportes `.Rmd`, dashboards Shiny y configuraciones.
- Cada análisis queda registrado con fecha, autor y mensaje descriptivo, facilitando la **auditoría** y la **reproducibilidad**.
- Si un cambio introduce un error, es posible revertir a una versión anterior con un solo comando.

2. GitHub como repositorio central del equipo BI

- GitHub actúa como el **único origen de verdad** (*single source of truth*) para todo el código de análisis.
- Los miembros del equipo clonan, actualizan y sincronizan sus cambios desde un mismo repositorio, eliminando la confusión de versiones locales dispersas.
- Ejemplo de estructura de repositorio BI con R:

```

proyecto-bi/
|-- .Rprofile
|-- renv.lock          # Dependencias reproducibles (renv)
|-- data/
|   |-- raw/           # Datos crudos (inmutables)
|   |-- processed/     # Datos transformados
|-- scripts/
|   |-- 01_extraccion.R
|   |-- 02_limpieza.R

```

```
| |-- 03_modelo.R
|-- reports/
| |-- informe_ventas.Rmd
| |-- dashboard/
| | |-- app.R          # Aplicacion Shiny
|-- tests/             # Pruebas unitarias (testthat)
|-- README.md
|-- .gitignore
```

3. Flujo de trabajo colaborativo con pull requests

El flujo típico de trabajo en equipo utilizando GitHub incluye:

1. **Crear una rama (*branch*)** para una tarea específica (nuevo modelo, corrección de reporte, limpieza de datos).
2. **Desarrollar y confirmar cambios (*commit*)** localmente, con mensajes descriptivos.
3. **Subir la rama a GitHub (*push*)** y abrir un **Pull Request (PR)**.
4. **Revisión de código (*code review*)** por parte de otros miembros del equipo, asegurando calidad y consistencia.
5. **Integración continua** mediante GitHub Actions que ejecutan los scripts de R automáticamente.
6. **Fusión (*merge*)** a la rama principal una vez aprobado y verificado.

4. Integración continua con GitHub Actions

GitHub Actions permite automatizar la ejecución de análisis de R directamente desde el repositorio:

```
# .github/workflows/analisis-bi.yml
name: Análisis BI automatizado

on:
  push:
    branches: [ main ]
  schedule:
    - cron: '0 6 * * 1'    # Todos los lunes a las 06:00 UTC

jobs:
  render:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
      - uses: r-lib/actions/setup-renv@v2
      - name: Renderizar reporte R Markdown
        run: Rscript -e 'rmarkdown::render("reports/informe_ventas.Rmd")'
```

```
- name: Subir reporte generado
  uses: actions/upload-artifact@v4
  with:
    name: informe-ventas
    path: reports/informe_ventas.html
```

Este flujo permite: - **Actualizar reportes automáticamente** cada semana o tras cada cambio en los datos. - **Ejecutar pruebas** (`testthat`) para validar que las transformaciones y modelos siguen funcionando. - **Notificar al equipo** si algún análisis falla después de un cambio.

5. Integración con RStudio / Posit

- RStudio (hoy Posit) incluye soporte nativo para Git, permitiendo realizar commits, push, pull y crear ramas desde la interfaz gráfica.
- **GitHub Copilot** puede asistir en la escritura de código R directamente desde RStudio.
- **Posit Cloud** y **Posit Connect** se integran con repositorios de GitHub para desplegar dashboards Shiny y reportes R Markdown automáticamente.

6. Gestión de dependencias reproducibles con renv

- El paquete **renv** crea un archivo `renv.lock` que registra las versiones exactas de cada paquete de R utilizado.
- Al incluir `renv.lock` en el repositorio de GitHub, todo el equipo (y la CI) puede restaurar el mismo entorno con `renv::restore()`.
- Esto elimina los problemas de “*en mi máquina funciona*” y garantiza reproducibilidad total.

7. Beneficios de integrar GitHub en la solución BI con R

Beneficio	Descripción
Trazabilidad total	Cada análisis, modelo y reporte tiene un historial completo de cambios.
Colaboración asíncrona	Varios analistas pueden trabajar en paralelo sin pisarse el trabajo.
Calidad del código	Las revisiones mediante PR mejoran la calidad y detectan errores a tiempo.
Automatización	GitHub Actions ejecuta análisis y genera reportes sin intervención manual.
Reproducibilidad	renv + Git garantizan que el análisis se puede replicar en cualquier momento.
Onboarding rápido	Un nuevo miembro del equipo clona el repositorio y tiene todo listo.

En definitiva, **GitHub no es solo un lugar donde almacenar código**: es una plataforma completa que potencia el trabajo en equipo, automatiza procesos y eleva la calidad y confiabilidad de una solución de Business Intelligence basada en R.

Documentación del trabajo BI con R Markdown y Markdown

En toda solución de Business Intelligence, el análisis de datos es solo una parte del ciclo. Para que los resultados generen valor real en la organización, es necesario **documentar, exponer y distribuir** el conocimiento de manera clara, accesible y reutilizable. Aquí es donde **R Markdown (.Rmd)** y **Markdown (.md)** juegan un papel central.

1. R Markdown: reportes dinámicos que integran código y narrativa

R Markdown permite combinar en un solo archivo: - **Texto explicativo** en Markdown, para describir objetivos, metodología, hipótesis y conclusiones. - **Código R ejecutable**, que se evalúa al renderizar el documento y produce resultados actualizados automáticamente. - **Salidas enriquecidas**: tablas, gráficos estáticos e interactivos, y valores calculados en línea.

Esto transforma un script de R tradicional en un **reporte BI completo y reproducible**. Cada vez que se renderiza el **.Rmd**, se actualizan todos los resultados con los datos más recientes, eliminando el trabajo manual de copiar y pegar números o gráficos en presentaciones o correos.

A continuación se muestra la **estructura típica de un archivo .Rmd** desglosada en sus tres componentes fundamentales:

1. Encabezado YAML — metadatos del documento El bloque YAML (delimitado por `---`) define el título, autor, fecha y formato de salida:

```
---
title: "Informe mensual de ventas"
author: "Equipo BI"
date: "`r Sys.Date()`"
output: html_document
---
```

2. Bloques de código R ejecutable El código R se escribe dentro de bloques cercados por ````${r}````. Al renderizar, estos bloques se ejecutan y sus resultados (tablas, gráficos, valores) se insertan en el documento final:

```
```${r} echo=FALSE, message=FALSE}
library(dplyr)
```

```
library(ggplot2)

ventas <- read.csv("data/ventas_mensuales.csv")
total_transacciones <- nrow(ventas)
ingreso_total <- sum(ventas$monto)

ventas %>%
 group_by(region) %>%
 summarise(Ventas = sum(monto)) %>%
 ggplot(aes(x = region, y = Ventas, fill = region)) +
 geom_col() +
 labs(title = "Ventas por región") +
 theme_minimal()
...

```

En este ejemplo, el código carga los datos, calcula dos métricas clave y genera un gráfico de barras por región. Las opciones `echo=FALSE` y `message=FALSE` ocultan el código fuente y los mensajes en el reporte final.

**3. Texto narrativo con valores calculados en línea** R Markdown permite insertar el resultado de cualquier expresión R directamente en el texto usando ``r código_r``:

```
En `r month.name[month(Sys.Date()) - 1]` se procesaron
`r format(total_transacciones, big.mark = ",")` transacciones,
con un ingreso total de **`r scales::dollar(ingreso_total)`**.
```

Al renderizar el informe, los fragmentos ``r ...`` se evalúan y se reemplazan por los valores reales (por ejemplo: “*En mayo se procesaron 12.483 transacciones, con un ingreso total de \$ 1.234.567*”), generando un reporte actualizado automáticamente sin copiar y pegar manualmente.

## 2. Diferencias y usos de .Rmd y .md en el proceso BI

Formato	Propósito principal	Contiene código ejecutable	Salida típica
.Rmd	Análisis y reportes dinámicos	Sí (bloques <code>```\${r}```</code> )	HTML, PDF, Word, PowerPoint, Dashboard
.md	Documentación estática y difusión	No	Archivo de texto plano visualizable en GitHub, GitLab, etc.

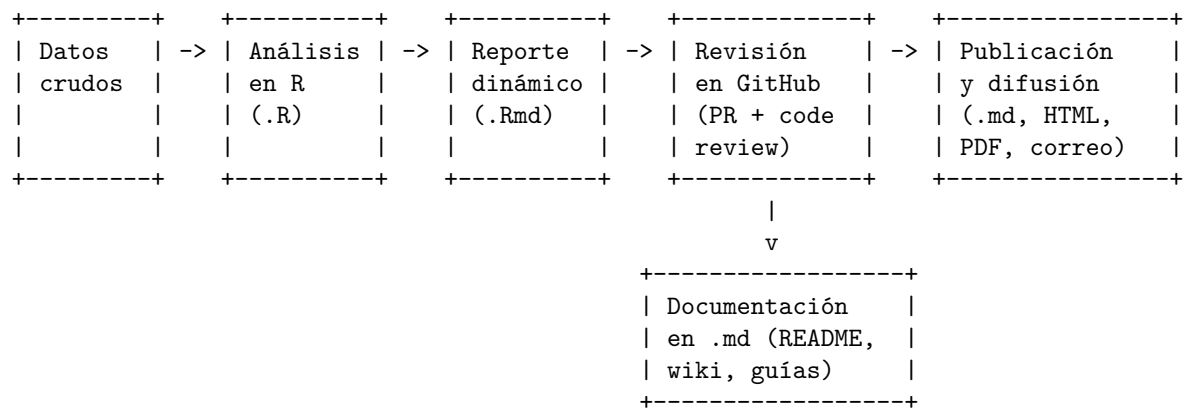
**Cuándo usar cada uno:**



- **.Rmd**: Para la fase de *análisis y generación de reportes*. Es el archivo fuente que contiene la lógica completa del análisis (código + explicación). Se renderiza para producir el entregable final.
- **.md**: Para la fase de *difusión y documentación permanente*. Una vez que el análisis está completo y validado, se puede generar o escribir un **.md** que presente los resultados de forma legible directamente en GitHub, sin necesidad de ejecutar código. También es el formato ideal para documentar procesos, acuerdos, guías de estilo y decisiones del equipo BI.

### 3. Flujo de documentación dentro del proceso BI

La documentación debe integrarse como una fase explícita dentro del flujo de trabajo BI:



1. **Los scripts .R** contienen la lógica de extracción, limpieza y modelado.
2. **Los archivos .Rmd** envuelven esa lógica en un reporte narrativo, explicando el *qué*, *por qué* y *cómo* de cada paso.
3. **La revisión en GitHub** mediante Pull Requests permite que el equipo valide tanto el código como la narrativa antes de publicar.
4. **Los archivos .md** (README, documentación de procesos, guías de estilo) mantienen la documentación viva y accesible para toda la organización.
5. **La publicación** puede ser automática (GitHub Actions renderiza el **.Rmd** y despliega el **.html** en GitHub Pages) o manual (se comparte el PDF por correo o se sube a una intranet).

### 4. GitHub Pages como plataforma de difusión

GitHub Pages permite publicar los reportes HTML generados desde **.Rmd** directamente como sitio web, sin necesidad de infraestructura adicional:

```
.github/workflows/deploy-report.yml
name: Desplegar reporte BI
```

```

on:
 push:
 branches: [main]
 paths:
 - 'reports/**/*.Rmd'

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: r-lib/actions/setup-r@v2
 - uses: r-lib/actions/setup-renv@v2
 - name: Renderizar y publicar
 run: |
 Rscript -e 'rmarkdown::render("reports/informe_ventas.Rmd")'
 mkdir -p docs
 cp reports/informe_ventas.html docs/index.html
 - uses: peaceiris/actions-gh-pages@v3
 with:
 github_token: ${ secrets.GITHUB_TOKEN }
 publish_dir: ./docs

```

Una vez configurado, el reporte estará disponible en <https://<organizacion>.github.io/<repositorio>/> y cualquier persona en la organización puede acceder a la última versión del análisis sin necesidad de instalar R ni ejecutar código.

## 5. Gestión de entornos reproducibles con `renv`

Un reporte R Markdown no es completamente reproducible si solo depende del código fuente. Las versiones de los paquetes de R instalados en la máquina del analista pueden diferir de las del servidor de CI o de las de otro miembro del equipo, provocando resultados inconsistentes o errores inesperados.

**renv** resuelve este problema creando un entorno aislado por proyecto que registra las versiones exactas de cada paquete en un archivo `renv.lock`. Este archivo se incluye en el repositorio de Git y permite que cualquier persona (o proceso automatizado) pueda restaurar el mismo entorno con un solo comando.

Para integrar **renv** en un proyecto BI con R Markdown:

1. **Inicializar `renv`** en la raíz del proyecto con `renv::init()`. Esto crea el archivo `renv.lock` y un directorio `renv/` con el código del paquete.
2. **Desarrollar el análisis** como de costumbre: instalar paquetes con `install.packages()`. `renv` detecta automáticamente los nuevos paquetes y actualiza `renv.lock` con `renv::snapshot()`.
3. **Confirmar `renv.lock` en Git** junto con el resto del código (`renv.lock` es un archivo de texto plano ideal para control de versiones).

4. **Restaurar el entorno** en cualquier otro equipo o en la CI con `renv::restore()`, que descarga e instala las versiones exactas registradas en `renv.lock`.

Ejemplo de workflow con **renv** en GitHub Actions:

```
steps:
 - uses: actions/checkout@v4
 - uses: r-lib/actions/setup-r@v2
 - uses: r-lib/actions/setup-renv@v2 # Ejecuta renv::restore() automáticamente
 - name: Renderizar reporte
 run: Rscript -e 'rmarkdown::render("reports/informe_ventas.Rmd")'
```

Gracias a **renv**, el reporte generado localmente, el que se genera en la CI y el que obtenga cualquier otro analista serán **idénticos en resultados**, eliminando de raíz los problemas de “*en mi máquina funciona*”.

## 6. Buenas prácticas de documentación BI con R Markdown

Práctica	Descripción
<b>Incluir metadatos</b>	Título, autor, fecha, versión y propósito del análisis en el encabezado YAML.
<b>Separar código y narrativa</b>	Explicar en lenguaje natural cada bloque de código; no asumir que el lector conoce R.
<b>Documentar supuestos</b>	Especificar qué filtros, umbrales o transformaciones se aplicaron y por qué.
<b>Incluir advertencias</b>	Indicar limitaciones de los datos, tamaño de muestra, o periodos no cubiertos.
<b>Versionar los .Rmd</b>	Almacenarlos en el repositorio Git junto con los .R para mantener la trazabilidad.
<b>Generar siempre una versión .md</b>	Resumir los hallazgos clave en un .md visible directamente en GitHub para consulta rápida.
<b>Automatizar la renderización</b>	Usar GitHub Actions para regenerar reportes periódicamente o ante cada cambio.
<b>Usar renv para entornos reproducibles</b>	Incluir <code>renv.lock</code> en el repositorio para que todo el equipo y la CI de GitHub Actions restauren el mismo entorno con <code>renv::restore()</code> , eliminando incompatibilidades entre versiones de paquetes.

## 7. Impacto en la organización

Una estrategia de documentación bien implementada con R Markdown y Markdown produce beneficios concretos:

- **Alta dirección y stakeholders:** Reciben reportes ejecutivos (HTML/PDF) con gráficos y conclusiones claras, sin necesidad de ver código.

- **Equipo de datos:** Dispone de documentación técnica (`.md` + `.Rmd`) que explica cada decisión analítica, facilitando el mantenimiento y la rotación de personal.
- **Audidores y cumplimiento:** Pueden rastrear exactamente qué datos y métodos se usaron en cada reporte, gracias al control de versiones de Git y la reproducibilidad de R Markdown.
- **Nuevos integrantes:** Se ponen al día rápidamente leyendo la documentación y ejecutando los reportes existentes.

En resumen, **documentar el trabajo BI en `.Rmd` y `.md` no es un paso opcional**: es el puente que conecta el análisis técnico con la toma de decisiones organizacional, garantizando que los resultados no se queden en el ordenador del analista sino que se difundan, comprendan y aprovechen en toda la empresa.

---

## Shiny: aplicaciones interactivas y dashboards para BI

### 1. ¿Qué es Shiny y por qué es relevante para BI?

**Shiny** es un paquete de R que permite construir **aplicaciones web interactivas** directamente desde R, sin necesidad de conocimientos de HTML, CSS o JavaScript. Para una solución de Business Intelligence, Shiny cubre un vacío crítico que R Markdown no puede llenar: la **interactividad en tiempo real**.

Mientras que R Markdown genera reportes estáticos (aunque actualizables), Shiny permite que el usuario final —analista, gerente o director— **interactúe con los datos y modelos** en vivo: ajuste parámetros, filtre fechas, explore escenarios y vea los resultados al instante, sin esperar a que un reporte se renderice de nuevo.

Shiny se integra de forma natural con el resto del ecosistema R: los mismos paquetes de manipulación (`dplyr`), visualización (`ggplot2`, `plotly`) y modelado se usan dentro de la aplicación, lo que evita la duplicación de lógica entre el análisis exploratorio y el dashboard final.

### 2. Shiny en el trabajo operativo: simulaciones estadísticas interactivas

En el día a día operativo, el equipo de datos necesita ejecutar análisis que respondan preguntas condicionales del tipo “¿qué pasa si...?”. Con Shiny, estas simulaciones se convierten en herramientas interactivas que cualquier miembro del equipo puede utilizar sin escribir código.

#### Ejemplo: Simulador de riesgo crediticio

Un analista de riesgos quiere evaluar cómo cambiaría la tasa de incobrabilidad si se modifican los umbrales de aprobación de créditos. Con Shiny puede construir una aplicación que:

```

UI: controles para los parámetros de simulación
ui <- fluidPage(
 titlePanel("Simulador de riesgo crediticio"),
 sidebarLayout(
 sidebarPanel(
 sliderInput("ingreso_min", "Ingreso mínimo (USD)",
 min = 500, max = 5000, value = 1500, step = 100),
 sliderInput("edad_min", "Edad mínima",
 min = 18, max = 70, value = 25),
 numericInput("max_deuda", "Deuda máxima permitida (USD)",
 value = 10000, min = 0, step = 1000),
 actionButton("simular", "Ejecutar simulación")
),
 mainPanel(
 h3("Resultados de la simulación"),
 verbatimTextOutput("resumen"),
 plotOutput("distribucion")
)
)
)

Server: lógica de simulación reactiva
server <- function(input, output) {
 observeEvent(input$simular, {
 candidatos <- read.csv("data/solicitudes.csv")

 filtrados <- candidatos %>%
 filter(ingresos >= input$ingreso_min,
 edad >= input$edad_min,
 deuda_total <= input$max_deuda)

 prob_incobrabilidad <- mean(filtrados$default_historico)
 total_aprobados <- nrow(filtrados)
 monto_total <- sum(filtrados$monto_solicitado)

 output$resumen <- renderPrint({
 cat("Resultados de la simulación:\n")
 cat(" Solicitudes que cumplen criterios:", total_aprobados, "\n")
 cat(" Monto total comprometido: USD", format(monto_total, big.mark = ","), "\n")
 cat(" Probabilidad estimada de incobrabilidad:",
 scales::percent(prob_incobrabilidad), "\n")
 })

 output$distribucion <- renderPlot({
 ggplot(filtrados, aes(x = monto_solicitado)) +
 geom_histogram(fill = "steelblue", bins = 30) +

```

```

 labs(title = "Distribución de montos solicitados (aprobados)",
 x = "Monto (USD)", y = "Frecuencia") +
 theme_minimal()
 })
 })
}

```

```
shinyApp(ui, server)
```

Esta aplicación permite al equipo operativo **ajustar parámetros en tiempo real** y ver al instante cómo cambian las métricas de riesgo, sin depender del analista que escribió el modelo para cada consulta.

Otros ejemplos operativos típicos con Shiny:

- **Simulador de inventarios:** ajustar niveles de stock de seguridad y ver el impacto en rotación y quiebres de stock.
- **Asignación de presupuesto publicitario:** modificar la distribución del gasto entre canales (digital, TV, radio) y observar el retorno estimado.
- **Segmentación dinámica de clientes:** cambiar los umbrales de RFM (recencia, frecuencia, monto) y ver cómo se reclasifican los clientes.

### 3. Shiny en el trabajo estratégico: tableros de control y dashboards

Para la toma de decisiones estratégica, los dashboards de Shiny proporcionan una **visión consolidada y actualizada** de los indicadores clave de negocio (KPIs), accesible desde cualquier navegador sin instalar software.

#### Ejemplo: Dashboard ejecutivo de ventas

Un director comercial necesita monitorear diariamente la evolución de las ventas, la rentabilidad por región y el cumplimiento de cuotas. Con Shiny y flexdashboard se puede construir un tablero ejecutivo:

```

title: "Dashboard ejecutivo de ventas"
output:
 flexdashboard::flex_dashboard:
 orientation: rows
 theme: cosmo

```{r setup, include=FALSE}
library(dplyr)
library(ggplot2)
library(plotly)
library(shiny)

ventas <- read.csv("data/ventas_diarias.csv")

```

```
...
```

Fila 1

```
### KPI - Ventas acumuladas
```

```
``{r}  
valueBox(  
  value = scales::dollar(sum(ventas$monto), prefix = "$"),  
  caption = "Ventas acumuladas del mes",  
  icon = "fa-chart-line"  
)  
...
```

```
### KPI - Margen promedio
```

```
``{r}  
margen <- mean(ventas$margen, na.rm = TRUE)  
valueBox(  
  value = scales::percent(margen),  
  caption = "Margen bruto promedio",  
  icon = "fa-percentage"  
)  
...
```

```
### KPI - Cumplimiento de cuota
```

```
``{r}  
cuota <- sum(ventas$monto) / 500000 # Cuota mensual: USD 500K  
valueBox(  
  value = scales::percent(cuota),  
  caption = "Cumplimiento de cuota mensual",  
  icon = "fa-bullseye",  
  color = ifelse(cuota >= 1, "success", "warning")  
)  
...
```

Fila 2

```
### Ventas por región
```

```
``{r}  
ventas %>%  
  group_by(region) %>%
```

```

    summarise(Total = sum(monto)) %>%
    plot_ly(x = ~region, y = ~Total, type = "bar", color = ~region) %>%
    layout(title = "Ventas por región", xaxis = list(title = ""),
           yaxis = list(title = "Total (USD)"))
...

### Evolución diaria

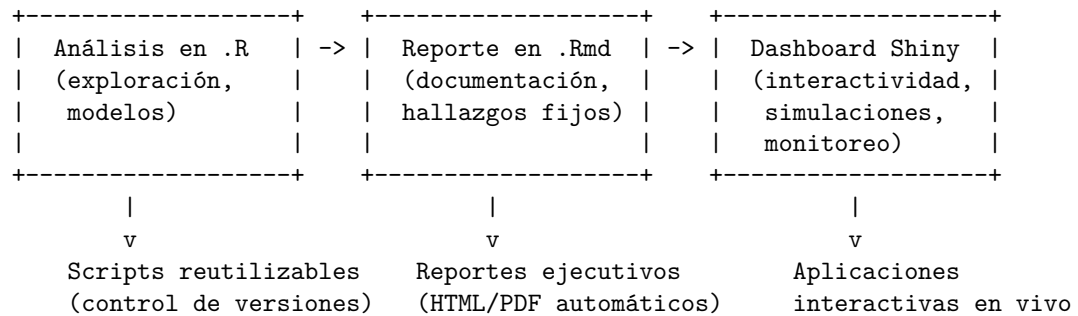
```{r}
ventas %>%
 group_by(fecha) %>%
 summarise(Diario = sum(monto)) %>%
 plot_ly(x = ~fecha, y = ~Diario, type = "scatter", mode = "lines+markers") %>%
 layout(title = "Ventas diarias del mes",
 xaxis = list(title = ""), yaxis = list(title = "USD"))
...

```

Este dashboard se despliega en un servidor (Shiny Server, Posit Connect o incluso gratis en shinyapps.io) y el equipo directivo accede a él mediante una URL, con datos actualizados cada vez que se recarga la página.

#### 4. Shiny en el ciclo BI: integración con el ecosistema existente

Shiny no reemplaza a R Markdown ni a los scripts .R, sino que los complementa dentro del mismo flujo de trabajo BI:



#### Buenas prácticas para integrar Shiny en el proceso BI:

Práctica	Descripción
<b>Reutilizar la lógica .R</b>	Extraer las funciones de manipulación y modelado en scripts .R separados; tanto .Rmd como Shiny los cargan con <code>source()</code> .
<b>Separar UI y server</b>	Mantener <code>ui.R</code> y <code>server.R</code> en archivos distintos para facilitar el mantenimiento y las revisión en PR.



Práctica	Descripción
<b>Usar <code>renv</code> también en Shiny</b>	Incluir el <code>renv.lock</code> para que el servidor de despliegue tenga las mismas versiones de paquetes que el entorno de desarrollo.
<b>Control de versiones</b>	Versionar el código de la aplicación Shiny en el mismo repositorio Git que los análisis y reportes.
<b>Automatizar el despliegue</b>	Usar GitHub Actions para desplegar la aplicación en Posit Connect o Shiny Server tras cada push a la rama principal.
<b>Diseñar para el usuario final</b>	Priorizar la usabilidad: controles claros, tiempos de respuesta rápidos y visualizaciones autoexplicativas.

## 5. Impacto de Shiny en la organización

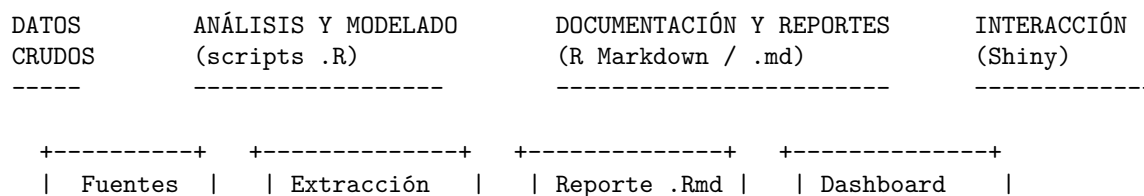
La incorporación de Shiny como capa interactiva de la solución BI transforma la relación del equipo de datos con el resto de la organización:

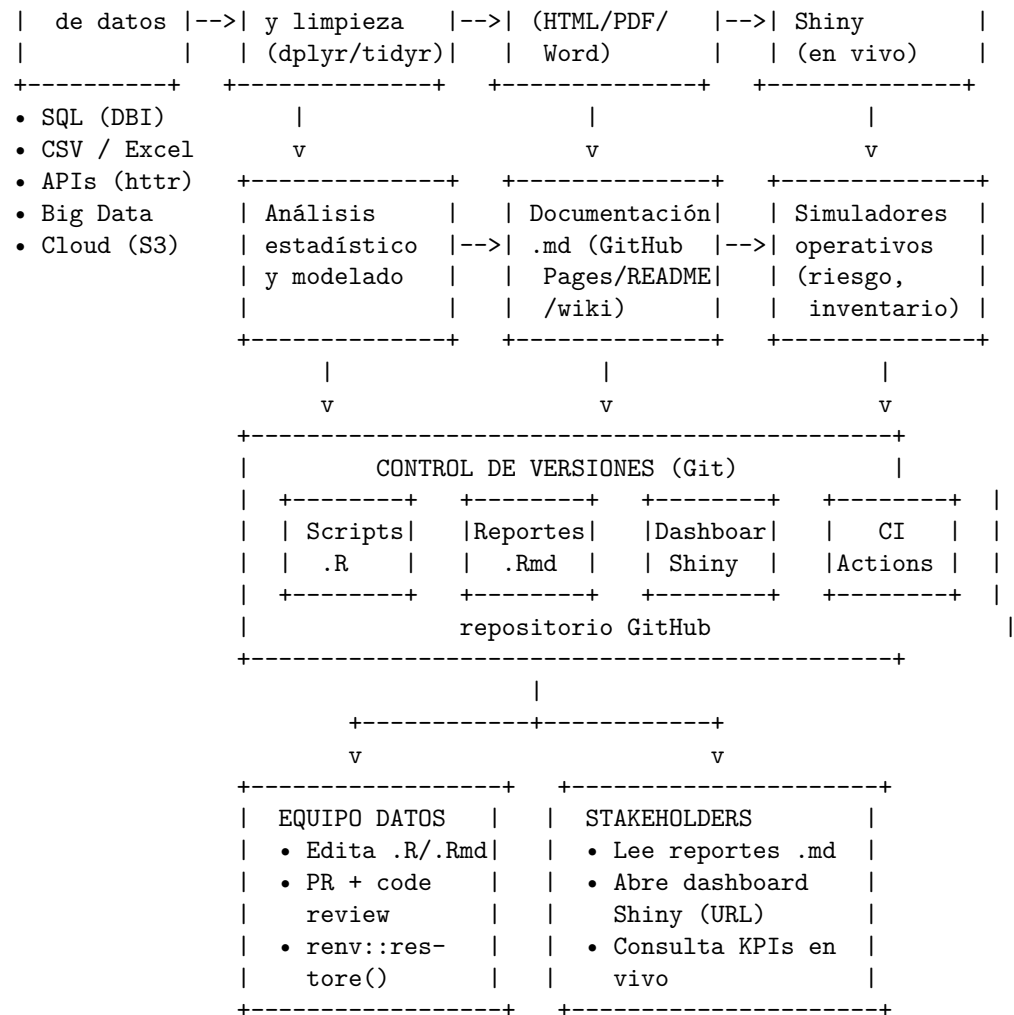
- **Equipo operativo:** Dispone de simuladores que le permiten explorar escenarios y tomar decisiones basadas en datos sin depender del analista para cada consulta.
- **Dirección estratégica:** Accede a dashboards en vivo con KPIs actualizados, sin esperar a que se genere un reporte periódico.
- **Equipo de datos:** Centraliza la lógica de negocio en una única base de código (scripts `.R`) que alimenta tanto los reportes estáticos como las aplicaciones interactivas, reduciendo la duplicación y las inconsistencias.

En definitiva, **Shiny convierte a R en una plataforma BI completa:** los reportes R Markdown documentan y comunican hallazgos, mientras que las aplicaciones Shiny permiten explorar, simular y monitorear en tiempo real, cubriendo desde el análisis operativo más granular hasta la toma de decisiones estratégica de alto nivel.

## Mapa completo del proceso BI con R

A continuación se presenta una visión integrada de todo el flujo de trabajo BI con R, articulando cada fase del proceso con las tecnologías, herramientas y buenas prácticas estudiadas en este documento. Este esquema unifica los conceptos presentados en las secciones anteriores en un solo mapa de referencia.





## Capas tecnológicas del ecosistema

Capa	Tecnología / Herramienta	Propósito
<b>Datos</b>	DBI, odbc, readr, httr, jsonlite, arrow	Conexión a fuentes: SQL, CSV, APIs, Big Data, cloud
<b>Transformación</b>	dplyr, tidyr, data.table	Limpieza, agregación y preparación de datos
<b>Análisis y modelado</b>	Modelos estadísticos nativos (regresión, series temporales, clustering, árboles)	Obtención de insights y predicciones

Capa	Tecnología / Herramienta	Propósito
<b>Visualización</b>	ggplot2, plotly, leaflet	Gráficos estáticos e interactivos, mapas
<b>Reportes dinámicos</b>	rmarkdown, flexdashboard	Documentos reproducibles con código y narrativa
<b>Documentación estática</b>	Markdown (.md)	README, wikis, guías visibles en GitHub
<b>Aplicaciones interactivas</b>	shiny	Dashboards ejecutivos y simuladores operativos
<b>Control de versiones</b>	Git + GitHub	Historial, colaboración, PRs, trazabilidad
<b>Entorno reproducible</b>	renv	renv.lock con versiones exactas de paquetes
<b>CI/CD</b>	GitHub Actions	Renderizado automático de .Rmd, despliegue de Shiny
<b>Publicación</b>	GitHub Pages, Posit Connect, shinyapps.io	Visibilidad web de reportes y dashboards

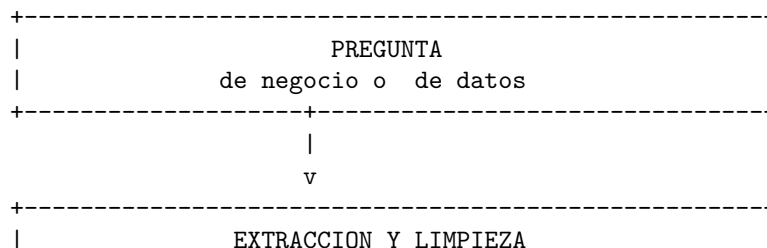
### Flujo de trabajo colaborativo por rol

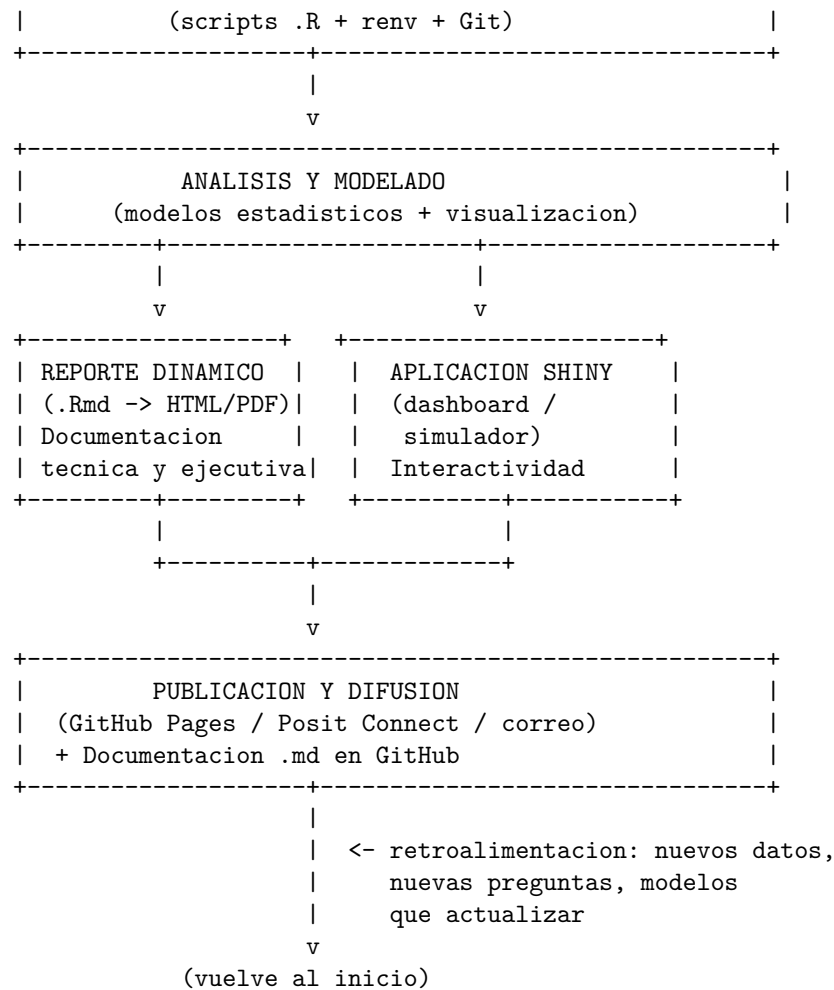
Cada perfil dentro de la organización interactúa con el ecosistema BI en distintos niveles:

ROL	ACCIÓN PRINCIPAL	HERRAMIENTAS QUE USA
---	-----	-----
Analista de datos	Escribe scripts .R, crea reportes .Rmd	RStudio, Git, renv
Científico de datos	Desarrolla modelos predictivos	R, ggplot2, caret, tidymodels
Ingeniero de datos	Automatiza extracciones y CI/CD	DBI, GitHub Actions, renv
Analista de negocio	Ejecuta simuladores, explora escenarios	Shiny (como usuario)
Director / Gerente	Consulta KPIs en dashboards en vivo	Shiny (navegador web)
Revisor / Auditor	Valida PRs, revisa historial de cambios	GitHub (code review)

### Ciclo de vida completo

El proceso no termina con la publicación: los hallazgos generan nuevas preguntas, los datos se actualizan y los modelos se refinan. El ciclo se reinicia constantemente:





Este es el ecosistema completo: un ciclo continuo donde los datos crudos se transforman en conocimiento accionable, documentado y accesible para toda la organización, respaldado por control de versiones, entornos reproducibles y automatización.

## Conclusión

R es una solución BI **poderosa, flexible y gratuita** que cubre todo el ciclo de vida del análisis de datos: desde la extracción y limpieza hasta la visualización, modelado predictivo, generación de reportes automatizados y creación de dashboards interactivos. Si bien herramientas como Power BI o Tableau ofrecen una experiencia más visual e inmediata, R destaca cuando se requiere **rigor estadístico, reproducibilidad, personalización avanzada y capacidad de**

**simulación interactiva**, convirtiéndolo en una opción ideal para equipos de datos que buscan una solución BI completa y escalable sin depender de software propietario.